

Output-Distribution Distillation for Contextual Sparsity Prediction in SwiGLU LLMs

Anonymous ACL submission

Abstract

Learned sparsity predictors accelerate large language model (LLM) inference by anticipating which neurons to skip, but a fundamental mismatch limits their effectiveness: predictors are trained per-neuron via binary cross-entropy while deployed under global top- k allocation that ranks neurons jointly across all layers. We bridge this gap with output-distribution Kullback–Leibler (KL) divergence, which couples all pruning decisions end-to-end through the model’s forward pass using Gumbel-Sigmoid relaxation. On LLaMA-3.1-8B, our predictor (<1% of base parameters) reduces perplexity by 14.5% over TEAL at 30% sparsity and renders post-hoc compensation networks redundant. Diagnostic analysis reveals a surprising finding: KL-optimized masks diverge 68% from magnitude-based targets, yet a target-swap ablation shows these discovered patterns, not end-to-end coupling per se, carry most of the quality signal. Cross-architecture evaluation on Mistral-7B and Qwen-2.5-14B confirms perplexity generalization, though knowledge-intensive tasks exhibit systematic regression at larger scales. We release our code and data at¹.

1 Introduction

Contextual activation sparsity, skipping neurons that contribute negligibly to each token, is an attractive approach to accelerating large language model (LLM) inference in SwiGLU architectures (Shazeer, 2020). Learned predictors (Liu et al., 2023; Gautam et al., 2026) train small networks to anticipate which neurons matter, but they universally rely on binary cross-entropy (BCE), treating each neuron as an independent classification target. At inference, however, these per-neuron scores feed TEAL (Liu et al., 2025), a global top- k mechanism that ranks neurons *jointly* across all

layers, retaining only those above a model-wide threshold. This creates a fundamental mismatch: predictors are optimized for local per-neuron accuracy but evaluated under global cross-layer allocation. The consequences of this misalignment for contextual sparsity prediction remain uninvestigated.

We close this gap by training sparsity predictors with output-distribution Kullback–Leibler (KL) divergence, which couples all masking decisions through the model’s forward pass. Under BCE, the gradient for neuron j depends only on j ’s own activation magnitude; under KL, the chain rule introduces a downstream Jacobian that couples each neuron to every other retained neuron in subsequent layers (§3.6). This allows the predictor to evaluate each neuron’s importance in context, considering how pruning one neuron affects downstream computation when other neurons are simultaneously pruned. Unlike MaskLLM (Fang et al., 2024), which applies KL to static N:M weight masks, our predictors generate contextual masks that vary per token and coordinate across layers through global allocation.

The results reveal a surprising picture of what drives quality in learned sparsity prediction. On LLaMA-3.1-8B (Grattafiori et al., 2024), our lightweight predictor (<1% of base parameters) trained with forward KL and Gumbel-Sigmoid relaxation (Jang et al., 2017) reduces perplexity (PPL) by 14.5% over TEAL at 30% sparsity and renders post-hoc compensation networks redundant. More revealingly, the KL objective discovers mask patterns that diverge 68% from magnitude-based targets, and a target-swap ablation isolates the source: BCE retrained on KL-derived targets recovers comparable perplexity (≤ 0.65 pp gap), suggesting that KL’s primary contribution is discovering better mask targets rather than providing end-to-end coupling during training. Cross-architecture evaluation on Mistral-7B-v0.3 (Jiang et al., 2023)

¹https://github.com/anonymous-nlp-2026-2/distill_sparse_swiglu

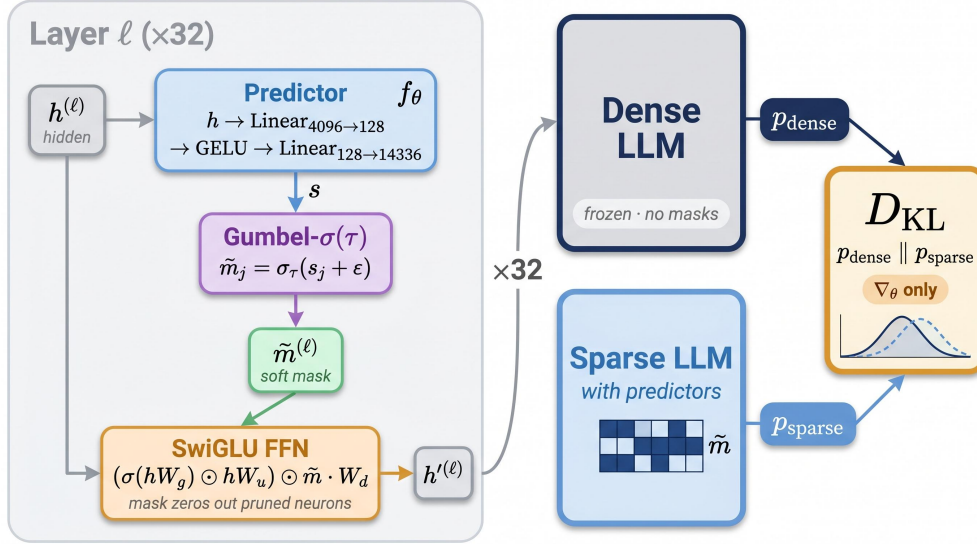


Figure 1: Method overview: a lightweight per-layer predictor produces importance scores trained via KL divergence with Gumbel-Sigmoid relaxation. At inference, scores feed TEAL’s global top- k allocation without compensation networks.

and Qwen-2.5-14B (Team, 2024) confirms perplexity gains, though knowledge-intensive tasks exhibit systematic regression, a structural limitation we trace to output-distribution optimization (§4).

Our contributions:

- Output-distribution alignment consistently outperforms per-neuron classification for learned sparsity, ranking first among five losses across two sparsity levels and three architectures;
- Per-layer controls reveal that the signal type gap (9.47 PPL) dominates the coupling breadth range (2.08 PPL) at 30% sparsity, suggesting what predictors optimize for matters more than how neurons interact during training (§4.5);
- Target-swap ablation shows KL-discovered mask patterns transfer quality to BCE, isolating the discovered targets, not the training dynamics, as the primary driver;
- Task-type analysis across three model families (7–14B) reveals a structural limitation: output-distribution optimization improves commonsense reasoning but degrades knowledge-intensive tasks.

2 Related Work

Activation Sparsity. Training-free methods exploit activation statistics directly: TEAL (Liu et al., 2025) applies global top- k by magnitude; CATS (Lee et al., 2024) adapts per-layer thresholds for SwiGLU; R-Sparse (Zhang et al., 2025) uses SVD projections; DIP (Federici et al., 2024)

combines dynamic input pruning with cache-aware masking; LaRoSA (Huang et al., 2025) applies layerwise rotations for structured sparsity with wall-clock speedup. Dhar et al. (Dhar et al., 2024) show SwiGLU exhibits lower natural sparsity than ReLU. Predictor-based methods learn input-dependent masks: DeJaVu (Liu et al., 2023) trains per-neuron BCE classifiers; ShadowLLM (Akhauri et al., 2024) uses per-layer MSE for intermediate coupling; FastForward (Gautam et al., 2026) adds a 67.2M-parameter compensation network; PowerInfer (Song et al., 2024a) exploits power-law activation frequencies for heterogeneous scheduling. MaskLLM (Fang et al., 2024) shares our recipe (frozen weights + learnable masks + KL), but targets static N:M weight sparsity via Gumbel-Softmax; our predictor generates *per-token* masks under global allocation that must generalize to unseen inputs. No prior work systematically compares loss functions for contextual activation sparsity prediction.

Compensation and Architecture. SPON (Xu et al., 2025) learns input-independent bias vectors via closed-form per-layer KL, absorbed into weights at deployment—a static per-layer correction weakly correlated with our contextual predictions (cosine 0.196; Appendix Q). ReLU Strikes Back (Mirzadeh et al., 2024) replaces SiLU with ReLU for >90% natural sparsity; TurboSparse (Song et al., 2024b) and ProSparse (Song et al., 2025) engineer extreme sparsity through

activation modifications; Haziza et al. (Haziza et al., 2025) demonstrate 2:4 semi-structured sparsity with hardware-native acceleration. MoC (Wu et al., 2025b) selectively activates top- K FFN channels using SwiGLU’s native gating during pretraining. All require retraining or architecture modification; our predictor operates on *unmodified* SwiGLU models.

Pruning, Distillation, and Routing. Wanda (Sun et al., 2024) and SparseGPT (Frantar and Alistarh, 2023) produce static weight masks via one-shot pruning; SliceGPT (Ashkboos et al., 2024) removes entire rows via orthogonal projection. KL-based distillation (Kim and Rush, 2016; Sanh et al., 2019; Gu et al., 2024; Agarwal et al., 2024) trains compact students; we apply this principle to train a mask predictor using Gumbel-Sigmoid (Maddison et al., 2017; Louizos et al., 2018) for differentiable discrete optimization. Expert Choice routing (Zhou et al., 2022) parallels TEAL’s global allocation; D2D-MoE (Szatkowski et al., 2024) converts dense SwiGLU layers into dynamic- k MoE, restructuring the network rather than predicting masks on the original architecture.

3 Method

We present an output-distribution KL framework for training contextual sparsity predictors in SwiGLU-based language models (Figure 1). The method replaces per-neuron binary classification (BCE) with an output-distribution alignment objective that couples all pruning decisions through the model’s forward pass and achieves competitive performance without auxiliary error compensation networks at inference.

3.1 Problem Formulation

Consider a pretrained transformer with L layers, each containing a SwiGLU feed-forward network (Shazeer, 2020). For layer ℓ , the FFN computes:

$$\mathbf{y}^{(\ell)} = (\sigma(\mathbf{x}^{(\ell)} \mathbf{W}_g^{(\ell)}) \odot \mathbf{x}^{(\ell)} \mathbf{W}_u^{(\ell)}) \mathbf{W}_d^{(\ell)}, \quad (1)$$

where $\mathbf{x}^{(\ell)} \in \mathbb{R}^d$ is the hidden state after attention ($d=4096$), σ denotes SiLU, $\mathbf{W}_g^{(\ell)}, \mathbf{W}_u^{(\ell)} \in \mathbb{R}^{d \times d_{\text{ff}}}$ and $\mathbf{W}_d^{(\ell)} \in \mathbb{R}^{d_{\text{ff}} \times d}$ are the gate, up-projection, and down-projection matrices, and $d_{\text{ff}}=14,336$.

Define the gate activation $\mathbf{g}^{(\ell)} = \sigma(\mathbf{x}^{(\ell)} \mathbf{W}_g^{(\ell)}) \in \mathbb{R}^{d_{\text{ff}}}$. Each element $g_j^{(\ell)}$ determines the contribution of neuron j to the output. We introduce a binary

mask $\mathbf{m}^{(\ell)} \in \{0, 1\}^{d_{\text{ff}}}$ that zeros out pruned neurons:

$$\mathbf{y}_{\text{sparse}}^{(\ell)} = (\mathbf{m}^{(\ell)} \odot \mathbf{g}^{(\ell)} \odot \mathbf{x}^{(\ell)} \mathbf{W}_u^{(\ell)}) \mathbf{W}_d^{(\ell)}. \quad (2)$$

Our goal is to learn a mask predictor $f_{\theta}^{(\ell)}(\mathbf{x}^{(\ell)}) \rightarrow \mathbf{s}^{(\ell)} \in (0, 1)^{d_{\text{ff}}}$ that produces per-neuron importance scores from $\mathbf{x}^{(\ell)}$. At inference, the scores are fed to a global top- k allocation mechanism (Liu et al., 2025) that distributes the sparsity budget across layers non-uniformly, retaining the $(1-\rho) \times L \times d_{\text{ff}}$ highest-scoring neurons model-wide for a target sparsity ratio ρ .

3.2 Predictor Architecture

Each layer ℓ has an independent two-layer MLP predictor:

$$f_{\theta}^{(\ell)}(\mathbf{x}^{(\ell)}) = \text{GELU}(\mathbf{x}^{(\ell)} \mathbf{W}_1^{(\ell)}) \mathbf{W}_2^{(\ell)}, \quad (3)$$

where $\mathbf{W}_1^{(\ell)} \in \mathbb{R}^{d \times d_b}$ and $\mathbf{W}_2^{(\ell)} \in \mathbb{R}^{d_b \times d_{\text{ff}}}$ with bottleneck dimension $d_b=128$. The predictor takes $\mathbf{x}^{(\ell)}$ (the hidden state after attention, before the FFN) as input, since this representation already encodes the token’s contextual dependencies from self-attention and thus provides the information needed to predict which FFN neurons will be important. The output is d_{ff} -dimensional logits $\mathbf{s}^{(\ell)}$, from which the mask is derived.

The bottleneck $d_b=128$ (0.9% of d_{ff}) balances expressiveness against overhead: sweeps over $d_b \in \{64, 128, 256\}$ show diminishing returns beyond 128, with $d_b=256$ adding $2 \times$ parameters for $<0.1\text{pp}$ PPL gain (Appendix B). We use GELU rather than ReLU to avoid dead predictor neurons during the early exploration phase when Gumbel noise dominates. Each predictor has $(4096 \times 128) + (128 \times 14336) = 2.37\text{M}$ parameters; with 32 independent predictors (one per layer), the total overhead is 75.9M, less than 1% of the 8B base. The per-layer independence is a deliberate design choice: cross-layer communication at inference would add sequential latency, and the K -curve analysis (§4.5) confirms that cross-layer gradient coupling during training provides modest benefit beyond the output-distribution signal itself.

3.3 Gumbel-Sigmoid Relaxation

The hard mask $m_j = \mathbf{1}[s_j > \text{threshold}]$ is non-differentiable, preventing gradient flow from the KL objective through the mask to the predictor parameters. We apply Gumbel-Sigmoid relaxation (Maddison et al., 2017; Louizos et al., 2018;

Jang et al., 2017) to produce differentiable soft masks:

$$\tilde{m}_j^{(\ell)} = \sigma_\tau \left(\log \frac{s_j^{(\ell)}}{1-s_j^{(\ell)}} + \varepsilon_1 - \varepsilon_2 \right), \quad \varepsilon_{1,2} \sim \text{Gumbel}(0, 1), \quad (4)$$

where $\sigma_\tau(x) = \text{sigmoid}(x/\tau)$ and τ is a temperature parameter. The Gumbel noise provides stochastic exploration of mask configurations during training, while the temperature controls sharpness: as $\tau \rightarrow 0$, $\tilde{m}_j$ approaches a binary decision.

We anneal τ linearly from 1.0 to 0.1 over training. At high temperature, the soft mask permits gradient flow through all neurons for broad exploration. As temperature decreases, the mask concentrates on confident decisions, approximating the hard allocation used at inference.

Comparison with Straight-Through Estimation. The straight-through estimator (STE) (Benio et al., 2013) applies a hard threshold in the forward pass and passes gradients through unchanged ($\partial m / \partial s = 1$). In our setting, STE produces discontinuous gradient signals: small parameter changes can flip many neurons simultaneously. The resulting oscillations in the penalty weight prevent convergence to the target sparsity. Gumbel relaxation eliminates this pathology and contributes a 25.2% improvement (PPL 15.77 $\rightarrow$ 11.79 at 50% sparsity), as we verify empirically in §4.

3.4 KL Divergence Training Objective

Let $\mathbf{z}_{\text{dense}}$ denote the output logits from a full dense forward pass, and $\mathbf{z}_{\text{sparse}}$ the logits from a forward pass with predictor-generated soft masks $\{\tilde{\mathbf{m}}^{(\ell)}\}_\ell$ applied at every layer. We minimize the forward KL divergence between the dense and sparse output distributions:

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}(p_{\text{dense}} \| p_{\text{sparse}}) = \sum_v p_{\text{dense}}^{(v)} \log \frac{p_{\text{dense}}^{(v)}}{p_{\text{sparse}}^{(v)}}, \quad (5)$$

where $p_{\text{dense}} = \text{softmax}(\mathbf{z}_{\text{dense}})$ and $p_{\text{sparse}} = \text{softmax}(\mathbf{z}_{\text{sparse}})$, summed over the vocabulary v and averaged over all token positions in the sequence. The base model is frozen; only the predictor parameters receive gradients.

We use forward KL (mass-covering) rather than reverse KL, as the goal is to preserve the dense model’s primary predictions rather than explore alternative modes. In sparsity prediction,

mode-covering is critical: false negatives (omitting a rarely but critically active neuron) cascade through downstream layers under global allocation, whereas false positives merely waste capacity; reverse KL’s mode-seeking concentrates on high-probability neurons and risks missing compositionally important activations (Wu et al., 2025a). §4 validates this choice empirically: forward KL outperforms reverse KL by 7.9% and Jensen–Shannon divergence (JSD) by 6.0% (at 50% sparsity; the gap narrows at 30%; Table 4). No temperature scaling ($T=1$) is applied (Hinton et al., 2015); we distinguish T from the Gumbel temperature τ (§3.3), which controls mask sharpness rather than logit scaling.

3.5 Geometric Penalty Schedule

We enforce a model-level sparsity target ρ via constrained optimization with $\bar{s} = \frac{1}{L} \sum_\ell \frac{1}{d_{\text{ff}}} \sum_j \tilde{m}_j^{(\ell)}$ denoting the average retained fraction:

$$\min_{\theta} \mathcal{L}_{\text{KL}} \quad \text{s.t.} \quad \bar{s} = 1 - \rho. \quad (6)$$

We relax this via a geometric penalty schedule:

$$\mathcal{L} = \mathcal{L}_{\text{KL}} + \lambda \cdot \text{clamp}(|\bar{s} - (1 - \rho)| - \epsilon, 0)^2, \quad (7)$$

where $\epsilon = 0.02$ is a deadzone margin that suppresses oscillation near the target (sensitivity in Appendix C). The penalty weight λ is updated via geometric doubling (doubled when sparsity exceeds target, halved otherwise), clamped to $[\lambda_{\text{floor}}, \lambda_{\text{max}}]$, reaching target sparsity within 40–110 steps across all tested regimes. This schedule avoids the instability of fixed- λ training: too low permits unconstrained masks; too high collapses the predictor to trivial uniform scores. The deadzone ϵ is essential—without it, the penalty oscillates around the target and interferes with KL minimization during the critical Gumbel annealing phase (Appendix C).

3.6 Gradient Analysis: BCE vs. KL

We contrast the gradient structure of BCE and KL to build intuition for how KL discovers divergent mask patterns (§4.4). This analysis is suggestive; empirical evidence in §4.4 shows KL-discovered mask targets drive inference quality.

Under BCE, the predictor trains against oracle masks $m_j^* = \mathbf{1}[|g_j| \geq \text{top-}k \text{ threshold}]$ derived from activation magnitudes. The gradient $\partial \mathcal{L}_{\text{BCE}} / \partial s_j = \sigma(s_j) - m_j^*$ depends only on neuron j ; each neuron is trained independently with no inter-neuron coupling. Under output-distribution

Table 1: Taxonomy of learned sparsity prediction methods along correction granularity (rows) and training signal coupling (columns). Our method occupies the Per-token $\times$ Coupled cell (organizational framework). Empty cells represent unexplored combinations; their relative merit is an open question.

	Decomposed (BCE)	Intermediate (MSE)	Coupled (KL)
Static	—	—	SPON, MaskLLM
Per-token	DejaVu, FastForward	ShadowLLM, D2DMoE	Ours

KL, the chain rule introduces a downstream Jacobian $\partial \mathbf{z}_{\text{sparse}} / \partial \tilde{\mathbf{m}}_j^{(\ell)}$ that couples neuron j ’s gradient to all other retained neurons in layers ℓ through L , because the sparse hidden state propagates through subsequent attention and FFN operations. This coupling allows KL to evaluate each neuron’s importance in the context of the full sparse computation, rather than against a fixed magnitude target.

Gradient signal-to-noise ratio (SNR) measurements² reveal that BCE maintains consistently higher SNR than KL throughout training ($2.3\times$ at convergence; Figure 7; extended analysis in Appendix H), indicating that KL’s quality advantage does not arise from cleaner gradient signals. Rather, KL’s end-to-end coupled gradient serves as a *search mechanism* that discovers compositionally superior mask patterns, even when the per-step signal is noisier than BCE’s decomposed updates. The target-swap ablation in §4.4 suggests that the discovered mask targets, not the coupling mechanism itself, drive inference-stage quality.

3.7 Two-Dimensional Taxonomy of Sparsity Prediction Methods

We organize prior work along two axes: *correction granularity* (how the mask adapts to input) and *training signal coupling* (whether the objective captures inter-neuron dependencies).

Our method combines per-token prediction with coupled KL training (Table 1). KL on output logits directly optimizes next-token prediction loss, which composes across layers; per-layer mean squared error (MSE) does not: a mask minimizing FFN output error at layer ℓ may amplify errors in layers $\ell+1 \dots L$, explaining its $1.7\times/9.6\times$ worse PPL than TEAL at 30%/50% (50%: s42 only; §4). Full taxonomy discussion in Appendix O.

² $\text{SNR}_\ell = \|\mathbb{E}[\nabla_{\theta_\ell} \mathcal{L}]\|_2^2 / \text{Var}[\nabla_{\theta_\ell} \mathcal{L}]$, expectation over mini-batches.

4 Experiments

4.1 Experimental Setup

Model and Predictor. We evaluate on LLaMA-3.1-8B (Grattafiori et al., 2024), which uses SwiGLU FFN blocks (Shazeer, 2020) with $d_{\text{model}} = 4096$ and $d_{\text{ff}} = 14336$ across 32 layers. The sparsity predictor is a two-layer MLP with 2.37M parameters per layer (75.9M total), implemented as Linear(4096 $\rightarrow$ 128) + GELU + Linear(128 $\rightarrow$ 14336), trained to produce per-neuron importance scores that minimize output-distribution divergence under sparsity.

Training. All predictors are trained for 1000 steps on WikiText-103 with sequence length 2048 and batch size 4 on a single NVIDIA RTX PRO 6000 Blackwell Server Edition GPU (96 GB), using AdamW (Loshchilov and Hutter, 2019) with learning rate 1×10^{-3} , weight decay 1×10^{-2} , and cosine decay. The KL predictor uses Gumbel-Sigmoid relaxation (Jang et al., 2017) with temperature $\tau = 1.0$ annealed to 0.1 over training.

Evaluation. We report perplexity on WikiText-2 (Merity et al., 2017) (seq_len=2048) and downstream accuracy on ARC-Challenge (25-shot), WinoGrande (5-shot), Massive Multitask Language Understanding (MMLU; 5-shot), Grade School Math 8K (GSM8K; 8-shot), HellaSwag (10-shot), and Physical Intuition QA (PIQA; 0-shot).

Inference Protocol. At inference, the predictor produces per-neuron scores that are passed to TEAL’s global top- k allocation mechanism (Liu et al., 2025), which distributes the sparsity budget across layers non-uniformly. We term this *predictor-TEAL*: the predictor replaces TEAL’s magnitude-based scoring while retaining its allocation strategy.

Baselines. We compare against: (1) **Vanilla TEAL** (Liu et al., 2025), which uses activation magnitudes directly³; (2) **BCE predictor** (DejaVu-style (Liu et al., 2023)), trained with per-neuron binary cross-entropy on magnitude-based oracle masks for 1000 and 2700 steps; and (3) **Uniform allocation**, which distributes the sparsity budget equally across layers.

³TEAL is deterministic; single-run evaluation suffices. KL/BCE predictors require multi-seed evaluation due to random initialization.

Table 2: Main results on LLaMA-3.1-8B. $\ast 2.71\times$ KL compute (§4.3). $\pm$: 3-seed; unmarked: s42. $\ddagger$ Actual sparsity: KL 28.5% vs. TEAL 29.3% at 30% (Appendix D).

Sparsity	Method	PPL $\downarrow$	ARC-c	WG	MMLU	GSM8K
0%	Dense	6.12	58.28	78.06	65.27	48.75
30%	TEAL uniform	10.82	39.33	65.04	45.86	8.79
	BCE global	11.25	41.2	66.0	36.3	6.0
	BCE uniform	15.09 $\pm$ 0.13	33.45	61.09	43.05	3.97
	MSE uniform	18.52 $\pm$ 1.68	35.8 $\pm$ 4.1	59.8 $\pm$ 2.6	30.7 $\pm$ 2.6	2.3 $\pm$ 0.7
	KL global (Ours) $\ddagger$	8.96 $\pm$ 0.02	43.8$\pm$3.6	70.2$\pm$0.9	35.7 $\pm$ 9.5	8.0 $\pm$ 0.7
	KL uniform $\ddagger$	8.81$\pm$0.02	43.7 $\pm$ 2.5	68.9 $\pm$ 0.4	40.7 $\pm$ 1.3	7.5 $\pm$ 0.1
50%	TEAL uniform	23.90	27.47	54.38	29.26	1.74
	BCE (2700 steps) $\ast$	28.13 $\pm$ 0.20	27.6 $\pm$ 0.6	51.8 $\pm$ 0.6	26.1 $\pm$ 0.5	2.1 $\pm$ 0.2
	MSE uniform	229.26	25.00	50.59	23.79	1.59
	KL global (Ours)	11.84$\pm$0.15	36.3$\pm$3.2	63.8$\pm$0.3	28.3 $\pm$ 0.7	2.7$\pm$1.3
	KL uniform	11.92 $\pm$ 0.07	33.8 $\pm$ 1.2	62.9 $\pm$ 0.1	28.0 $\pm$ 0.6	2.2 $\pm$ 0.1

4.2 Main Results

Table 2 presents the primary comparison. Uniform allocation preserves downstream performance best; global allocation is analyzed to understand allocation-quality tradeoffs (§4.6). At 30% sparsity, output-distribution KL under uniform allocation achieves PPL 8.81 $\pm$ 0.02 (3-seed), a 14.5% reduction over TEAL at matched actual sparsity (8.81 vs. 10.30, both at 28.5%; Appendix D). Uniform allocation yields stable downstream (MMLU 40.7 $\pm$ 1.3%, 3-seed; Appendix F), while global top- k exhibits high seed variance (35.7 $\pm$ 9.5%) from late-layer starvation (L31 retention 28.4%, Appendix E; §4.6).

Task-Type Split. KL shows directional improvements on commonsense reasoning (ARC-c, WG, HellaSwag, PIQA: 4/4 wins at 30%; Appendix I) but underperforms on knowledge tasks (MMLU, GSM8K: 0/2). With three seeds, 95% CIs overlap for individual tasks; we treat PPL ($\sigma=0.02$, non-overlapping) as the primary metric. The KL predictor learns non-uniform per-layer sparsity, protecting middle layers (L14–18 at 14–18% vs. 32% mean; Appendix U). Constraint interventions (Appendix T) confirm a task-dependent trade-off: removing the learned valley improves MMLU +3.4pp but reduces ARC-c –1.5pp, consistent with localized factual associations (Geva et al., 2023). A C4-trained control (Appendix V) does not alleviate the regression, suggesting the deficit is structural to output-distribution optimization rather than training data composition.

At 50%, the PPL advantage widens (11.92 vs. 23.90, 50% reduction) and generalizes to C4 (Appendix D), but all methods collapse to near-random on MMLU ($\leq 29\%$) and GSM8K ($\leq 2.7\%$); no allocation strategy recovers at this capacity limit.

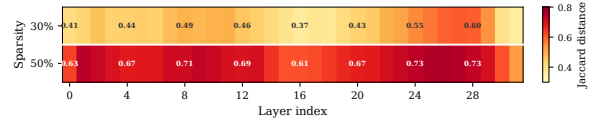


Figure 2: Per-layer Jaccard distance between KL-derived and magnitude-based masks at 30% and 50% sparsity (64 samples, 131K tokens). Divergence peaks in layers 24–28 (>0.73 at 50%).

4.3 Extended Training Analysis

At $2.71\times$ KL compute budget (2700 steps, matched wall-clock on a single RTX PRO 6000: ~ 46 min BCE vs. ~ 45 min KL), BCE achieves PPL 28.13 $\pm$ 0.20 (3-seed), still $2.39\times$ worse than KL (11.79). The gap is systematic, not due to underfitting: BCE reaches *higher* oracle F1 (0.613 vs. 0.540) with 139% worse PPL, confirming KL optimizes for distributional fidelity rather than magnitude agreement. This dissociation between oracle agreement and downstream quality is a key finding: better per-neuron classification accuracy does not translate to better output preservation under global allocation.

4.4 Mask Pattern Analysis

KL-derived masks diverge from magnitude oracles: per-layer Jaccard distance averages 68% at 50% (64 samples, 131K tokens; Figure 2), with cross-seed variation (36.3%) confirming systematic divergence (Appendices F.1, M). Despite this, KL achieves PPL 11.79 vs. BCE’s 28.13 $\pm$ 0.20, confirming that distributional fidelity, not magnitude agreement, produces better masks. Divergence follows a U-shaped per-layer pattern, peaking in late layers (0.70–0.75 at 50%).

Target-Swap Ablation: Target Quality vs. Gradient Coupling. To disentangle whether KL’s advantage stems from its coupled gradient pathway or from the mask patterns it discovers, we train a standard BCE predictor on KL-derived oracle masks instead of magnitude-based masks (C1 target-swap ablation). Table 3 reports results under TEAL uniform allocation at matched sparsity.

C1 matches KL within 0.79pp at 50% and ≤ 1.18 pp at 30%⁴, while magnitude-trained BCE yields $2.4\times$ worse PPL (Table 2). KL-discovered mask patterns thus *transfer* to simpler training (PPL within 0.79pp), though C1 binarizes soft logits, dis-

⁴s42 oracle from earlier checkpoint (gap 0.65pp); s123/s456 use seed-matched converged oracles (≤ 0.03 pp gap).

Table 3: Target-swap ablation (C1): BCE on KL-derived masks vs. KL predictor (same seed), TEAL uniform. All seeds show max $\Delta \leq 1.70\text{pp}$ at 30%; max $\Delta \leq 0.79\text{pp}$ at 50% (s42). [†]**Provenance:** s42 oracle extracted from an earlier training run (not seed-matched); s123/s456 use seed-matched retrained oracles (Appendix P).

Sparsity	Seed	Method	PPL↓	ARC-c	WG	MMLU	GSM8K
30%	s42 [†]	C1	9.45	40.27	67.40	42.30	7.58
		KL	8.82	40.78	68.59	40.6	7.35
	s123	C1	8.84	44.71	69.77	39.92	6.82
		KL	8.83	45.31	69.38	39.4	7.51
	s456	C1	8.82	44.71	69.93	41.04	6.60
		KL	8.79	45.05	68.75	42.0	7.58
50%	s42	C1	11.96	32.17	63.14	27.79	2.05
		KL	11.96	32.59	62.35	27.92	1.97

Table 4: Ablation on LLaMA-3.1-8B (WikiText-2 PPL↓; uniform at 30%, global at 50%). $\pm$: 3-seed; unmarked: s42. [‡]STE: 20.8% actual sparsity at 30% target; loss effect ($\Delta\text{PPL}=20.4$) is $5.1\times$ relaxation effect ($\Delta\text{PPL}=3.98$). [§]MSE 50%: 2/3 seeds collapsed to $\sim 5\%$ actual sparsity; s42 only (Appendix G).

Configuration	30% PPL↓	50% PPL↓	Δ_{50}
<i>KL-based variants</i>			
Forward KL + Gumbel (Ours)	8.81$\pm$0.02	11.79	—
JSD (symmetric) + Gumbel	8.90 $\pm$ 0.02	12.54	+0.75
Reverse KL + Gumbel	9.26 $\pm$ 0.07	12.80	+1.01
Forward KL + STE	— [‡]	15.77	+3.98
<i>Per-layer training</i>			
Per-layer MSE	18.52 $\pm$ 1.68	229.26 [§]	+217.47
Per-layer KL ($K=1$)	9.05 $\pm$ 0.02	34.85	+23.06
Per-layer KL ($K=4$)	8.89 $\pm$ 0.01	44.23	+32.44
<i>BCE-based variants</i>			
BCE (no compensation)	15.71	27.97	+16.18
BCE + Gumbel (no penalty)	—	32.22	+20.43
BCE + Gumbel + Penalty	—	38.02	+26.23
BCE + STE	—	60.54	+48.75

carding ranking information that may matter for global top- k allocation. The transfer result has a practical implication: once KL-derived oracles are extracted, simpler BCE predictors can be distilled from them without end-to-end KL training, trading a one-time oracle extraction cost for faster per-model deployment. BCE+Gumbel (32.22) underperforms standard BCE (27.97), indicating that Gumbel relaxation hurts when the target is a fixed binary mask rather than a distributional objective—the stochastic exploration explores suboptimal regions that the decomposed BCE loss cannot recover from (Appendix P).

4.5 Ablation Studies

Table 4 isolates the contribution of each design choice at 30% and 50% sparsity.

Relaxation and Divergence. At 50%, Gumbel-Sigmoid outperforms STE (Bengio et al., 2013)

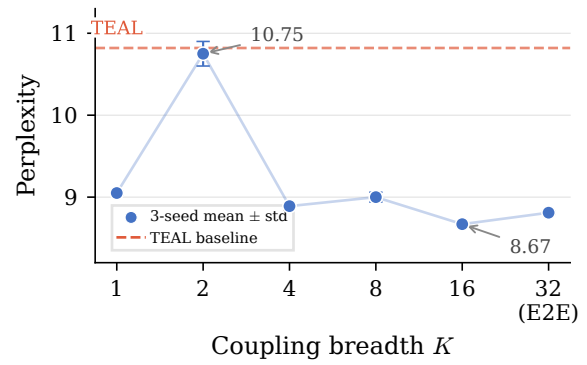


Figure 3: Coupling breadth K vs. perplexity (30%, TEAL uniform, 3-seed mean $\pm$ std). $K=32$ denotes end-to-end training. $K=2$ degradation reflects insufficient per-layer updates (~ 62 vs. ~ 250 at $K=8$).

by 25.2%; forward KL outperforms JSD (+6.0%) and reverse KL (+7.9%), consistent with mode-covering (Gu et al., 2024). At 30%, forward KL (8.81) and JSD (8.90) are near-identical, both outperforming reverse KL (9.26) and MSE (18.52 $\pm$ 1.68), suggesting the divergence measure matters less than optimizing through the output distribution (§4.5).

Per-Layer KL: Signal Type vs. Coupling Scope.

To isolate cross-layer coupling, we train per-layer KL predictors masking K random layers per step, computing KL on the full output without cross-layer gradients. The K -curve (Figure 3) shows per-layer KL ($K=1$) achieves 9.05 $\pm$ 0.02, within 2.7% of E2E (8.81), while per-layer MSE degrades to 18.52 $\pm$ 1.68. The signal type gap⁵ (9.47 PPL) exceeds the coupling breadth range (2.08 PPL; full K -curve in Appendix W), suggesting output-distribution alignment contributes more than coupling scope, though confounded by train-inference mismatch (§S). A $K=4$ control partially deconfounds: PPL gaps close while commonsense gaps persist, suggesting task-dependent coupling (Appendices S–R).

Compensation Redundancy. FastForward-style compensation (Gautam et al., 2026) (+67.2M params) provides no benefit (Appendices J–B.2); SPON bias correction is orthogonal (Appendix Q). Predictor capacity (Appendix B) and temperature schedule (Appendix B.1) show minimal sensitivity.

⁵“Signal type” denotes the optimization target (output distribution vs. intermediate activations), which also entails gradient flow and landscape differences not independently controlled.

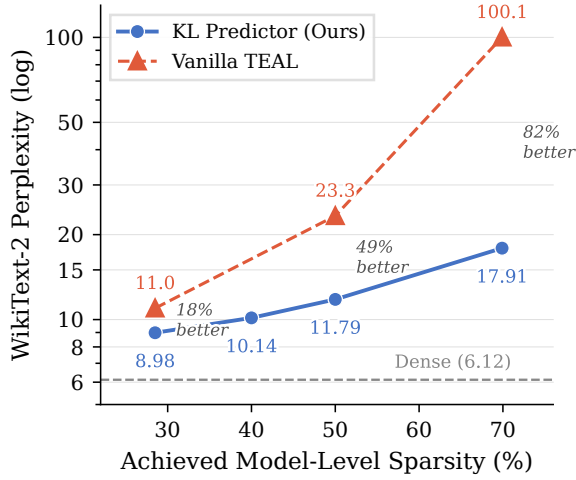


Figure 4: PPL scaling with sparsity (global allocation). KL’s advantage over TEAL grows with sparsity.



Figure 5: Per-layer neuron retention under global top- k at 30%, 50%, and 70% sparsity. Dashed lines: uniform baselines. Late layers (L24–31) are starved (28.4% at L31 even at 30%).

4.6 Sparsity Scaling and Regime Analysis

Allocation Behavior. Table 14 (Appendix L) reveals non-monotonic allocation preference: uniform is best at $\leq 30\%$ ($7.3\times$ smaller MMLU σ), global at 35–40%, both collapse at $\geq 45\%$. The non-monotonicity arises because global top- k concentrates budget in high-importance middle layers (L14–18, Figure 5), which helps at moderate sparsity but starves late layers at low sparsity (L31 retains only 28.4% at 30%). Uniform allocation avoids this pathology at the cost of over-allocating to unimportant layers, a trade-off that favors uniform when the total budget is sufficient.

Cross-Architecture and Scale. On Mistral-7B-v0.3 (Jiang et al., 2023), PPL generalizes (-17.3% , 3-seed; Appendix N) with directional common-

sense improvements and tied MMLU; on Qwen-2.5-14B (Team, 2024), PPL generalizes but MMLU and GSM8K decline substantially (Appendix K). The knowledge regression is consistent across all three architectures (8B MMLU -5.2pp , 14B GSM8K 32.2 ± 5.3 vs. 52.1), suggesting a systematic limitation of output-distribution KL rather than an architecture-specific artifact. We hypothesize this stems from KL’s mode-covering property: the predictor learns to preserve the dense model’s high-probability outputs (commonsense patterns distributed across many tokens) at the expense of rare but critical activations that encode factual associations in specific layers (Geva et al., 2023). The pipeline adds 6.4% latency overhead absent sparse kernels (Appendix A).

5 Conclusion

We systematically compared five training objectives for learned sparsity prediction in SwiGLU LLMs and found output-distribution alignment to be the primary design axis: forward KL outperforms JSD, reverse KL, BCE, and MSE, with per-neuron objectives degrading by $1.7\text{--}19\times$ in perplexity. The signal type gap (9.47 PPL between per-layer KL and MSE) exceeds the coupling breadth range (2.08 PPL across $K \in \{1, \dots, 32\}$), though confounded by train–inference mismatch; a target-swap ablation confirms that KL-discovered mask patterns transfer to simpler BCE training (PPL gap $\leq 0.65\text{pp}$), indicating that *what* to mask matters more than *how* the gradient flows. The KL predictor ($<1\%$ of base parameters) reduces TEAL’s perplexity by 14.5% at 30% on LLaMA-3.1-8B, generalizing to Mistral-7B (-17.3%) and Qwen-2.5-14B, while rendering compensation ($+67.2\text{M}$ params) redundant. Gains are task-type dependent: KL improves commonsense reasoning (4/4 tasks) but shows knowledge-task regression confirmed structural via C4 domain control.

Sparse kernel integration for wall-clock speedup and task-aware allocation strategies that balance commonsense and knowledge preservation are the most pressing open directions.

Limitations

Our findings are bounded by four limitations. **Sparsity sweet spot.** 30% is the lowest-degradation level; at 50%, all methods collapse on reasoning tasks, and a separate predictor must be retrained per target. **Knowledge-intensive regression.** KL con-

sistently underperforms on MMLU and GSM8K across scales (8B: −5.2pp MMLU; 14B: −19.9pp GSM8K), suggesting output-distribution objectives systematically sacrifice reasoning-critical pathways. **Evaluation scope.** Open-ended generation (e.g., MT-Bench) and contexts beyond 2048 tokens remain untested. **Baseline coverage.** The 30% isocompute comparison lacks a BCE 2700-step baseline; WikiText-2 evaluation shares provenance with the training corpus (WikiText-103), partially mitigated by C4 out-of-distribution (OOD) results (Appendix D). MSE uses identical hyperparameters to KL; its gradient magnitude is $\sim 5\times$ larger, and performance may improve with a dedicated learning rate.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. 2024. [On-policy distillation of language models: Learning from self-generated mistakes](#). In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Yash Akhauri, Ahmed F AbouElhamayed, Jordan Dotzel, Zhiru Zhang, Alexander M Rush, Safeen Huda, and Mohamed S Abdelfattah. 2024. [Shad-owlm: Predictor-based contextual sparsity for large language models](#). In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. 2024. [Slicegpt: Compress large language models by deleting rows and columns](#). In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. [Estimating or propagating gradients through stochastic neurons for conditional computation](#). *arXiv preprint arXiv:1308.3432*.

Nobel Dhar, Bobin Deng, Md Romyull Islam, Kazi Fahim Ahmad Nasif, Liang Zhao, and Kun Suo. 2024. [Activation sparsity opportunities for compressing general large language models](#). *arXiv preprint arXiv:2412.12178*.

Gongfan Fang, Hongxu Yin, Saurav Muralidharan, Greg Heinrich, Jeff Pool, Jan Kautz, Pavlo Molchanov, and Xinchao Wang. 2024. [Maskllm: Learnable semi-structured sparsity for large language models](#). In *Advances in Neural Information Processing Systems (NeurIPS)*.

Marco Federici, Davide Belli, Mart van Baalen, Amir Jalalirad, Andrii Skliar, Bence Major, Markus Nagel, and Paul Whatmough. 2024. [Efficient llm inference](#)

[using dynamic input pruning and cache-aware masking](#). In *Proceedings of Machine Learning and Systems (MLSys)*.

Elias Frantar and Dan Alistarh. 2023. [Sparsegpt: Massive language models can be accurately pruned in one-shot](#). In *Proceedings of the International Conference on Machine Learning (ICML)*.

Aayush Gautam, Mukul Gagrani, Junyoung Park, Mingu Lee, Chiris Lott, and Narasimha Reddy. 2026. [Fast forward: Accelerating llm prefill with predictive ffn sparsity](#). *arXiv preprint arXiv:2602.00397*.

Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. 2023. [Dissecting recall of factual associations in auto-regressive language models](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12216–12235.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aieleen Lakber, Aimin Chen, Ait Mokhtar Salma, and Ajay Menon. 2024. [The llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2024. [Minillm: Knowledge distillation of large language models](#). In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Daniel Haziza, Timothy Chou, Dhruv Choudhary, Luca Wehrstedt, Francisco Massa, Jiecao Yu, Geonhwa Jeong, Supriya Rao, Patrick Labatut, and Jesse Cai. 2025. [Accelerating transformer inference and training with 2:4 activation sparsity](#). *arXiv preprint arXiv:2503.16672*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. [Distilling the knowledge in a neural network](#). In *NeurIPS 2014 Deep Learning Workshop*.

Fangcheng Huang and 1 others. 2025. [LaRoSA: Accelerating large language model inference with layerwise rotated sparse activation](#). *arXiv preprint arXiv:2507.01299*.

Eric Jang, Shixiang Gu, and Ben Poole. 2017. [Categorical reparameterization with gumbel-softmax](#). In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, and Lucile Saulnier. 2023. [Mistral 7b](#). *arXiv preprint arXiv:2310.06825*.

Yoon Kim and Alexander M. Rush. 2016. [Sequence-level knowledge distillation](#). In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. 2024. Cats: Contextually-aware thresholding for sparsity in large language models . <i>arXiv preprint arXiv:2404.08763</i> .	754
James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. 2025. Training-free activation sparsity in large language models . In <i>Proceedings of the International Conference on Learning Representations (ICLR)</i> .	755
Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. 2023. Deja vu: Contextual sparsity for efficient llms at inference time . In <i>Proceedings of the International Conference on Machine Learning (ICML)</i> .	756
Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization . In <i>Proceedings of the International Conference on Learning Representations (ICLR)</i> .	757
Christos Louizos, Max Welling, and Diederik P. Kingma. 2018. Learning sparse neural networks through l_0 regularization . In <i>Proceedings of the International Conference on Learning Representations (ICLR)</i> .	758
Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. 2017. The concrete distribution: A continuous relaxation of discrete random variables . In <i>Proceedings of the International Conference on Learning Representations (ICLR)</i> .	759
Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models . In <i>Proceedings of the International Conference on Learning Representations (ICLR)</i> .	760
Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. Relu strikes back: Exploiting activation sparsity in large language models . In <i>Proceedings of the International Conference on Learning Representations (ICLR)</i> .	761
Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer . <i>Journal of Machine Learning Research</i> , 21(140):1–67.	762
Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter . <i>arXiv preprint arXiv:1910.01108</i> .	763
Noam Shazeer. 2020. Glu variants improve transformer . <i>arXiv preprint arXiv:2002.05202</i> .	764
Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. 2025. Prosparse: Introducing and enhancing intrinsic activation sparsity within large language models . In <i>Proceedings of the International Conference on Computational Linguistics (COLING)</i> .	765
Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024a. Powerinfer: Fast large language model serving with a consumer-grade gpu . In <i>Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)</i> .	766
Yixin Song, Haotong Xie, Zhengyan Zhang, Bo Wen, Li Ma, Zeyu Mi, and Haibo Chen. 2024b. Turbo sparse: Achieving llm sota performance with minimal activated parameters . <i>arXiv preprint arXiv:2406.05955</i> .	767
Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2024. A simple and effective pruning approach for large language models . In <i>Proceedings of the International Conference on Learning Representations (ICLR)</i> .	768
Filip Szatkowski, Bartosz Wójcik, Mikołaj Piórczyński, and Simone Scardapane. 2024. Exploiting activation sparsity with dense to dynamic-k mixture-of-experts conversion . In <i>Advances in Neural Information Processing Systems (NeurIPS)</i> , volume 37.	769
Qwen Team. 2024. Qwen2.5 technical report . <i>arXiv preprint arXiv:2412.15115</i> .	770
Taiqiang Wu, Chaofan Tao, Jiahao Wang, Runming Yang, Zhe Zhao, and Ngai Wong. 2025a. Rethinking kullback-leibler divergence in knowledge distillation for large language models . In <i>Proceedings of the International Conference on Computational Linguistics (COLING)</i> .	771
Tong Wu, Yutong He, Bin Wang, and Kun Yuan. 2025b. Mixture-of-channels: Exploiting sparse FFNs for efficient LLMs pre-training and inference . <i>arXiv preprint arXiv:2511.09323</i> .	772
Haotian Xu, Jiannan Yang, Tian Gao, Tsui-Wei Weng, and Tengfei Ma. 2025. Resting neurons, active insights: Robustify activation sparsity for large language models . In <i>Proceedings of the International Conference on Machine Learning (ICML)</i> .	773
Zhenyu Zhang, Zechun Liu, Yuandong Tian, Harshit Khaitan, Zhangyang Wang, and Steven Li. 2025. R-sparse: Rank-aware activation sparsity for efficient llm inference . In <i>Proceedings of the International Conference on Learning Representations (ICLR)</i> .	774
Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-experts with expert choice routing . In <i>Advances in Neural Information Processing Systems (NeurIPS)</i> , volume 35.	775

A Inference Efficiency

Table 5 reports inference throughput on a single NVIDIA RTX PRO 6000 Blackwell Server Edition GPU (96 GB; batch size 1, sequence length 2048, fp16, averaged over 100 forward passes).

Table 5: Inference latency on LLaMA-3.1-8B (RTX PRO 6000 Blackwell, batch=1, seq=2048, fp16, 100 forward passes). Overhead is relative to dense inference.

Method	Sparsity	ms/fwd	tok/s	Overhead
Dense	0%	119.11±0.57	17,195	—
KL predictor-only	30%	126.64±0.70	16,172	+6.3%
KL predictor-only	50%	126.78±0.31	16,154	+6.4%
Predictor + compensation	30%	129.94±0.75	15,761	+9.1%
Predictor + compensation	50%	129.71±0.53	15,789	+8.9%

The full KL-sparse pipeline adds 6.4% overhead to dense inference (126.78 vs. 119.11 ms/forward), of which the predictor’s 32-layer forward pass contributes 3.17 ms (2.7%); the remainder arises from mask application and sparse FFN indexing. Eliminating the compensation network saves an additional ~2.7% latency overhead and 67.2M parameters (the per-layer GEMV required for output rescaling); output-distribution KL thus provides both quality and efficiency advantages over BCE-based approaches. The overhead is near-identical at 30% and 50% sparsity because the predictor runs the same computation regardless of target sparsity; only the top- k threshold changes.

Note that the current implementation uses unstructured element-wise masks, which do not translate to wall-clock speedup without sparse kernel support. The pipeline adds ~6.4% overhead; net acceleration requires kernel-level sparse computation optimizations (e.g., Triton block-sparse, cuSPARSE) that are beyond the scope of this work. Our contribution is prediction quality and inference simplification (removing the compensation network), not end-to-end latency reduction.

B Predictor Bottleneck Sensitivity

Table 6 reports the effect of predictor bottleneck dimension d_b on PPL at 50% sparsity under both TEAL global allocation (the paper’s primary inference protocol) and per-token evaluation.

Under TEAL global allocation, increasing d_b beyond 128 provides no benefit (PPL slightly worsens), while per-token evaluation shows monotonic improvement (10.16→7.93). This divergence indicates that the global top- k allocation mechanism, not predictor capacity, is the binding constraint:

Table 6: Predictor bottleneck sensitivity at 50% sparsity (WikiText-2 PPL↓). TEAL global is the primary inference protocol; per-token shown for reference.

d_b	Params	TEAL global	Δ	Per-token
64	38.2M (0.48%)	12.076	+0.9%	10.16
128	75.9M (0.95%)	11.963	—	9.218
256	151.5M (1.89%)	12.105	+1.2%	8.586
512	302.5M (3.77%)	12.315	+2.9%	7.93

additional predictor capacity produces better per-neuron scores, but the global ranking and budget distribution cannot exploit them. The $d_b=128$ configuration (75.9M params, <1% of base model) is therefore optimal for the TEAL inference protocol.

B.1 Temperature Schedule Sensitivity

Table 7: Temperature schedule sensitivity at 50% sparsity (WikiText-2 PPL↓, TEAL global).

τ Schedule	PPL↓
Fixed $\tau=0.5$	11.861
Linear 1.0 → 0.1 (default)	11.963
Fixed $\tau=0.1$	12.072

Temperature schedule has minimal impact (<1.8% PPL variation). The linear annealing default is a conservative choice; even removing annealing entirely (fixed $\tau=0.5$) yields comparable performance.

B.2 Training Convergence

Table 8: Training convergence at 50% sparsity (WikiText-2 PPL↓, TEAL global).

Steps	PPL↓
200	15.158
400	12.538
600	12.046
800	11.907
1000	11.906

Training converges rapidly: 95% of the quality gap closes by step 600, and performance saturates by step 800 ($\Delta < 0.001$ between 800 and 1000 steps). Our default of 1000 steps provides a conservative margin.

C Geometric Penalty Schedule Dynamics

Table 9 reports the convergence of the geometric penalty schedule across sparsity targets. The geometric doubling pattern (1 → 2 → 4 → 8 → ...)

enables λ to reach steady state within ~ 10 steps; the deadzone ($\epsilon = 0.02$) then stabilizes λ once actual sparsity is within $\pm 2\%$ of the target.

Table 9: Geometric penalty schedule convergence across sparsity targets (margin $\epsilon = 0.02$, 1000 training steps). Deviation is $|\text{actual} - \text{target}| / \text{target}$.

Target ρ	$\lambda_{\max}$	Final λ	Actual sparsity	Deviation
30%	5000	5000 (cap)	0.285	5.0%
40%	5000	625	0.398	0.5%
50%	1000	8	0.504	0.8%
70%	1000	1000 (cap)	0.688	1.7%

At 50% sparsity, the SwiGLU gate distribution already clusters near the target, so λ remains low (8); the constraint is nearly satisfied by the initial predictor outputs. At 30% and 70%, the gap between the gate prior and target is larger, driving λ to its respective caps. The 40% target converges to $\lambda = 625$ with only 0.5% deviation; this precision reflects a regime where the target is reachable without λ saturation.

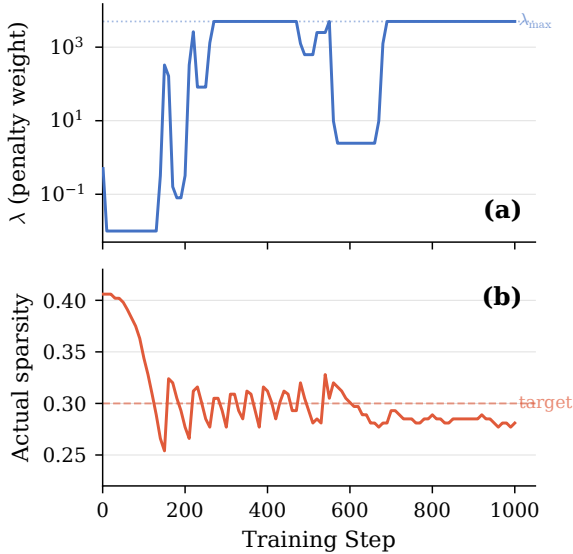


Figure 6: Lagrangian controller dynamics for 30% sparsity target ($\lambda_{\max} = 5000$). Left axis (log scale): penalty weight λ rises from 0.01 to its cap once actual sparsity undershoots the target at $\sim$ step 150, exhibiting bang-bang oscillations before saturating. Right axis: actual sparsity (solid) converges from the natural density (≈ 0.41) toward the target (dashed) and stabilizes near 0.28.

Temperature τ is annealed linearly from 1.0 to 0.1 over 1000 steps. A mild loss increase in the final ~ 100 steps is expected: as masks become more binary ($\tau \rightarrow 0$), discrete rounding noise slightly increases KL divergence, consistent with conver-

gence toward the hard-mask inference regime.

The controller reliably drives actual sparsity to within 5% of the target across all tested regimes, with λ spanning three orders of magnitude (8 to 5000) via geometric doubling. For the 30% target, actual sparsity is $\approx 28.5\%$ (5% deviation). This does not affect conclusions: TEAL at 28.5% actual sparsity yields PPL 10.30 (vs. 10.97 at exact 30%), a 0.67 PPL difference negligible relative to the KL vs. TEAL gap (8.96 vs. 10.97).

D Out-of-Distribution Evaluation

To address concerns about data homogeneity between training (WikiText-103) and evaluation (WikiText-2), we evaluate on C4 (Raffel et al., 2020) validation set under TEAL global allocation.

Table 10: Out-of-distribution evaluation on C4 validation (PPL $\downarrow$). 30% KL reports 2-seed mean $\pm$ std; other entries use seed 42.

Sparsity	Method	C4 PPL $\downarrow$
0%	Dense	9.21
30%	KL predictor (Ours)	16.03$\pm$0.01
50%	KL predictor (Ours)	18.40
	TEAL vanilla	38.75

At the 30% sparsity level, the KL predictor achieves C4 PPL 16.03 $\pm$ 0.01 (2-seed, global allocation, $1.74\times$ dense), confirming OOD generalization. At 50%, the KL predictor achieves 52.5% relative advantage over TEAL (18.40 vs. 38.75), comparable to the 50% advantage on WikiText-2 (11.92 vs. 23.90), confirming that KL-trained predictions generalize across distributions.

Uniform Allocation. Under uniform allocation at 30%, C4 PPL across three seeds is 18.22 (s42), 16.37 (s123), 16.36 (s456), yielding mean 16.98 $\pm$ 1.07 ($1.84\times$ dense). The cross-seed variance is substantially higher than in-domain: CV=6.3% on C4 vs. 0.2% on WikiText-2 (PPL 8.81 $\pm$ 0.02), driven by seed 42’s elevated C4 PPL (18.22 vs. 16.37 for other seeds). Global allocation’s cross-layer budget flexibility partially compensates for domain shift (PPL 16.03 global vs. 16.98 uniform), consistent with the Limitations observation that the predictor partially overfits to training-domain activation patterns.

Downstream Task Robustness. Table 11 compares downstream performance when the predictor generates masks from C4 text (OOD) vs. WikiText

(in-domain), both at 30% uniform allocation (seed 42). All six benchmarks show $\Delta \leq 1.1\text{pp}$, confirming that predictor masks are robust to calibration domain shift.

Table 11: Downstream task robustness to calibration domain shift (KL 30% uniform, seed 42). WikiText: predictor masks from in-domain text; C4: masks from OOD text. Metrics: acc_norm (ARC-c, HS, PIQA), acc (WG, MMLU), exact_match (GSM8K).

Calibration	ARC-c	WG	MMLU	GSM8K	HS	PIQA
WikiText	40.8	68.6	40.6	7.4	70.9	74.8
C4 (OOD)	40.4	69.3	41.4	6.3	70.7	74.0
Δ	-0.4	+0.7	+0.8	-1.1	-0.2	-0.8

E Per-Layer Retention Analysis

Figure 5 (main text) shows per-layer neuron retention under global top- k allocation. Late layers (L24–31) are consistently starved due to long-tailed activation distributions, with retention dropping to 28.4% (L31) even at 30% sparsity. At 70%, starvation extends to mid-layers (L10–12: 16.7–20.1%), explaining the transition to the capacity-limited regime.

F Training Stability

Three-seed evaluation ($s \in \{42, 123, 456\}$) at 50% sparsity under uniform allocation yields PPL 11.92 ± 0.07 (CV=0.6%); training is stable across random initializations. Downstream metrics are similarly stable: ARC-c $33.8 \pm 1.2\%$, WinoGrande $62.9 \pm 0.1\%$, MMLU $28.0 \pm 0.6\%$, GSM8K $2.2 \pm 0.1\%$.

Under global allocation at 50%, MMLU is $28.3 \pm 0.7\%$; the near-random performance is capacity-limited rather than seed-sensitive ($\sigma=0.7\text{pp}$ vs. 9.5pp at 30% global). The low standard deviation across both allocation strategies indicates that the KL training landscape is well-conditioned for this predictor architecture, with low sensitivity to initialization.

F.1 Cross-Seed Mask Consistency

To assess whether KL predictors discover consistent mask patterns across initializations, we compute pairwise Jaccard distances among three independently trained KL predictors (s42/s123/s456) at 30% uniform allocation (100 sequences, WikiText-2 validation).

The mean cross-seed Jaccard distance is 0.363 (IoU = 0.637), substantially lower than the KL-

vs-magnitude divergence (0.46–0.68), confirming that KL-discovered patterns form a distinct cluster from magnitude-based masks. Per-layer divergence follows a U-shaped pattern: lowest at embedding-adjacent layers (L0: 0.33), peaking at early layers (L1: 0.41), with a mid-network valley (L12–14: 0.33). The random baseline Jaccard distance is 0.462.

While specific mask configurations vary moderately across seeds (Jaccard distance 36.3%), the resulting PPL is highly stable (8.81 ± 0.02 , CV=0.2%), suggesting multiple near-equivalent optima in the activation mask space. The cross-seed divergence (36.3%) is substantially lower than the KL-vs-magnitude divergence (46–68%), confirming that KL discovers effective patterns that cluster distinctly from magnitude-based selection.

G MSE Baseline Configuration

The MSE predictor uses identical architecture (two-layer MLP, bottleneck $d_b=128$), training data (WikiText-103, 1000 steps), optimizer (AdamW, $\text{lr}=1 \times 10^{-3}$, weight decay 1×10^{-2} , cosine decay), Gumbel-Sigmoid mask generation (τ : 1.0 $\rightarrow$ 0.1), and adaptive Lagrangian sparsity control as the KL and BCE predictors. The only difference is the loss target: per-layer $\|\mathbf{W}_d^{(\ell)}(\tilde{\mathbf{m}}^{(\ell)} \odot \mathbf{g}^{(\ell)} \odot \mathbf{x}^{(\ell)} \mathbf{W}_u^{(\ell)}) - \mathbf{y}_{\text{dense}}^{(\ell)}\|_2^2$, replacing the end-to-end KL divergence. This controlled comparison isolates the effect of output-distribution alignment: all three predictors see identical data in identical order with identical architectures; only the loss function differs.

MSE uses $\lambda_{\text{max}}=1000$ (vs. KL’s 5000) because MSE’s per-layer gradient magnitude is approximately $5\times$ larger than KL’s end-to-end gradient, requiring proportionally less penalty pressure to achieve comparable sparsity control. Both reach their target sparsity within 110 steps. At 50% target, MSE converges to the target sparsity for seed 42; however, with seed 123, sparsity reaches only 5.1% under the same λ_{max} cap (eval PPL=15,789, random-level), indicating high sensitivity to initialization, a failure mode not observed with KL-based losses. At 30%, MSE achieves 27.7% actual sparsity for s42 (gap 2.3pp). The 3-seed MSE mean (18.52 ± 1.68 ; s42=20.43, s123=17.73, s456=17.39) is still $1.7\times$ worse than TEAL (10.82), confirming that the performance deficit stems from the loss function, not insufficient sparsity pressure.

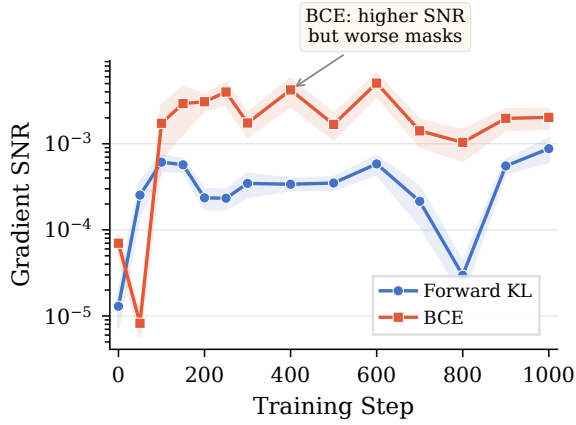


Figure 7: Gradient SNR trajectory across 1000 training steps. BCE (orange) maintains consistently higher SNR than KL (blue) from step 100 onward ($2.3\text{--}17\times$), yet KL produces superior masks (Table 2), suggesting that target alignment matters more than gradient clarity for mask quality.

H Extended SNR Analysis

Figure 7 plots gradient SNR across the full 1000-step training for both KL and BCE predictors, measured at 14 checkpoints (LLaMA-3.1-8B, C4) with 95% bootstrap confidence intervals. From step 100 onward, BCE maintains consistently higher SNR than KL ($2.3\text{--}17\times$ depending on training phase), with the gap narrowing toward convergence ($2.3\times$ at step 1000: 2.026×10^{-3} vs. 8.78×10^{-4}). Despite KL’s noisier gradient signal, KL-trained predictors produce substantially better masks (Table 2), confirming that *target alignment* matters more than gradient clarity for mask quality. BCE’s higher SNR reflects its simpler per-neuron classification objective, which concentrates gradients on individual neuron decisions; KL’s coupled gradient distributes signal across all mask decisions jointly, yielding lower SNR but discovering compositionally superior mask patterns (§3.6).

I Extended Downstream Evaluation

Table 12 reports HellaSwag and PIQA across sparsity levels and allocation strategies.

At 30%, the KL predictor retains 86.3% of dense HellaSwag accuracy (71.07 ± 0.54 vs. 82.35, 3-seed) and 92.6% of PIQA (74.95 ± 0.25 vs. 80.90), confirming moderate degradation on non-reasoning tasks. At 50%, global allocation outperforms uniform on HellaSwag (+2.29pp), consistent with the budget-concentration benefit observed for PPL in the capacity-limited regime (§4.6); PIQA shows a smaller gap (+0.38pp).

Table 12: Extended downstream benchmarks (LLaMA-3.1-8B). HellaSwag: 10-shot normalized acc; PIQA: 0-shot normalized acc. KL 30% uniform: 3-seed mean $\pm$ std; others: seed 42.

Sparsity	Method	HellaSwag	PIQA
0%	Dense	82.35	80.90
30%	KL uniform	71.07$\pm$0.54	74.95$\pm$0.25
	TEAL uniform	63.31	72.80
	BCE isocompute	62.81	74.21
50%	KL global	58.95	69.48
	KL uniform	56.66	69.10
	TEAL uniform	39.35	63.66

J Compensation Training Convergence

All compensation networks (Gautam et al., 2026) were trained for 1000 steps ($\text{lr}=10^{-4}$, batch size 4, seq length 2048). Training loss plateaus by step 700 across all valid runs: 30% s42 (final loss 0.70), 30% s456 (0.68), 50% s42 (1.07), with fluctuations remaining within 0.08 loss units for the final 300 steps. The compensation network converges but provides no quality improvement over the KL predictor alone, confirming that the redundancy result reflects predictor self-sufficiency rather than insufficient compensation training.

K Qwen-2.5-14B Detailed Results

Table 13: Qwen-2.5-14B at 30% nominal sparsity. Actual sparsity differs: KL 29.6% (3-seed), TEAL 29.3% (target 30%). **Bold**: best sparse per column per group. BCE collapses to 6.4% native sparsity (§3.6), so forced global top- k is used.

Method	PPL↓	ARC-c	WG	MMLU	GSM8K	HS	PIQA
Dense	5.19	67.58	81.06	79.77	87.57	84.20	81.94
<i>Global top-k allocation</i>							
KL (Ours)	7.62	57.17	72.06	63.72	48.82	75.01	77.37
TEAL	8.73	56.83	72.69	71.83	49.51	70.51	74.70
BCE	8.76	57.59	72.61	71.67	53.07	53.43	68.28
<i>Uniform allocation</i>							
KL (Ours)	7.90	53.50	72.82	63.55	32.2 $\pm$ 5.3	72.68$\pm$0.2	76.01$\pm$1.1
TEAL	8.01	55.29	69.85	66.34	52.1	67.72	74.70

GSM8K: 3-seed (s42/s123(v4)/s456).

Under global allocation, BCE’s HellaSwag drops to 53.43% (-17pp vs. TEAL), indicating that the magnitude-based predictor’s mask quality degrades severely when combined with non-uniform budget distribution at 14B scale.

L Full Allocation Analysis

Table 15 extends Table 14 with all downstream metrics.

Table 14: Allocation summary (LLaMA-3.1-8B). $\pm$: 3-seed; *high seed variance.

Sparsity	Allocation	PPL $\downarrow$	MMLU
30%	Global	8.96 ± 0.02	$35.7 \pm 9.5^*$
	Uniform	8.81 ± 0.02	40.7 ± 1.3
50%	Global	11.84 ± 0.15	28.3 ± 0.7
	Uniform	11.92 ± 0.07	28.0 ± 0.6

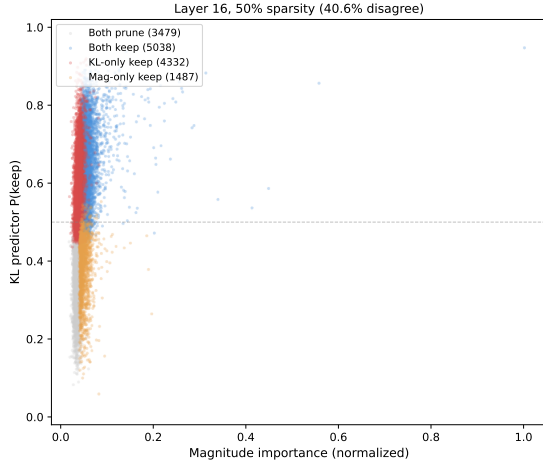


Figure 8: KL vs. magnitude importance scores for layer 16 neurons at 50% sparsity. 41% of neurons fall in disagreement quadrants where one method selects a neuron the other discards.

M Mask Pattern Visualization

Figure 2 (main text) shows per-layer Jaccard distance between KL-derived and magnitude-based masks at 30% and 50% sparsity. Figure 8 shows per-neuron importance scores at layer 16. Figure 9 shows binary mask comparisons at layers 2, 16, and 28.

N Cross-Architecture Comparison

Table 16 evaluates on Mistral-7B-v0.3 (Jiang et al., 2023) (GQA + sliding window, dense PPL 5.52).

The KL PPL advantage ratio is consistent across architectures (within 8% at $\leq 50\%$ sparsity). Table 17 reports downstream task accuracy at 30% sparsity on Mistral-7B-v0.3 (3-seed). KL improves PPL (-17.3%) and commonsense tasks (WG $+6.7\text{pp}$) but MMLU is tied (43.3% vs. 43.5%); ARC-c shows high seed variance ($\sigma=6.4\text{pp}$); GSM8K reverses.

O Taxonomy Discussion

Correction granularity. *Static* methods (Xu et al., 2025) learn a fixed bias vector per layer that

does not depend on the input token. *Per-token* methods (Liu et al., 2023; Gautam et al., 2026; Akhauri et al., 2024; Szatkowski et al., 2024) produce a unique mask for each input position, conditioning on the token’s hidden state at each layer.

Training signal coupling. *Decomposed* objectives (BCE) treat each neuron independently, optimizing per-element classification against an oracle mask. *Intermediate* objectives (MSE on hidden states) capture local coupling within a single layer but not end-to-end output effects. *Coupled* objectives (KL on output distribution) propagate gradients through the full forward pass; each mask decision’s gradient reflects its joint effect with all other masks on the final distribution.

SPON (Xu et al., 2025) distills a per-layer bias via KL but cannot adapt to per-input neuron importance (static $\times$ coupled). DeJaVu (Liu et al., 2023) and FastForward (Gautam et al., 2026) achieve per-token granularity but use BCE, where each neuron’s gradient is independent of other mask states. D2DMoE (Szatkowski et al., 2024) provides intermediate coupling (MSE on hidden states) but not end-to-end output effects. We define *output-distribution alignment* as whether the training loss optimizes a quantity that composes across layers. KL on output logits directly optimizes next-token prediction loss, which does not suffer from error composition; per-layer MSE minimizes reconstruction error independently at each layer, but layer-wise optima do not compose: a mask minimizing FFN output error at layer ℓ may amplify errors in layers $\ell+1 \dots L$. The SNR analysis (§3.6) corroborates this: KL achieves the best masks despite $2.3\times$ lower SNR than BCE, while MSE produces the worst masks. The framework predicts that losses operating on output-level quantities should outperform intermediate-level losses, a hypothesis consistent with our MSE results and testable with alternative output-level objectives.

P Oracle Mask Provenance

C1’s oracle masks are extracted from a converged KL predictor and binarized via hard threshold (logits > 0 , equivalent to $\sigma > 0.5$); they reflect one specific training trajectory (optimizer state, data order, Gumbel noise). The s42/s123/s456 consistency ($\leq 1.70\text{pp}$) provides partial evidence of robustness, but does not rule out trajectory-dependent confounds.

Table 15: Full allocation analysis: global top- k vs. uniform across sparsity levels (LLaMA-3.1-8B). Entries with $\pm$: 3-seed mean $\pm$ std ($s \in \{42, 123, 456\}$); others: seed 42. *High MMLU seed variance (individual seeds span 25.3–43.5%).

Sparsity	Allocation	PPL↓	ARC-c	WG	MMLU	GSM8K
25%	Global	8.17	45.4	70.6	46.6	11.5
	Uniform	8.23	46.5	71.8	44.0	11.0
30%	Global	8.96 $\pm$ 0.02	43.8$\pm$3.6	70.2$\pm$0.9	35.7 $\pm$ 9.5*	8.0$\pm$0.7
	Uniform	8.81$\pm$0.02	43.7 $\pm$ 2.5	68.9 $\pm$ 0.4	40.7$\pm$1.3	7.5 $\pm$ 0.1
35%	Global	9.31	38.9	67.6	40.4	7.7
	Uniform	9.44	38.7	66.5	37.8	4.3
40%	Global	10.14	39.5	67.5	39.5	6.6
	Uniform	10.76	39.6	63.1	35.5	3.6
45%	Global	10.80	39.2	64.6	30.4	3.0
	Uniform	10.95	37.6	63.6	30.6	2.7
50%	Global	11.84$\pm$0.15	36.3$\pm$3.2	63.8$\pm$0.3	28.3$\pm$0.7	2.7$\pm$1.3
	Uniform	11.92 $\pm$ 0.07	33.8 $\pm$ 1.2	62.9 $\pm$ 0.1	28.0 $\pm$ 0.6	2.2 $\pm$ 0.1

Table 16: Cross-architecture comparison (WikiText-2 PPL↓). Adv = TEAL / KL ratio. **Note:** LLaMA results use global top- k allocation; Mistral results use uniform allocation. Mistral 30%: 3-seed mean $\pm$ std; other rows: seed 42.

	Mistral-7B			LLaMA-8B		
	KL	TEAL	Adv	KL	TEAL	Adv
30%	6.93$\pm$0.19	8.38	1.21 $\times$	8.96	10.97	1.22 $\times$
50%	8.65	18.31	2.12 $\times$	11.79	23.31	1.98 $\times$
70%	13.33	61.72	4.63 $\times$	17.91	100.11	5.59 $\times$

Table 17: Mistral-7B-v0.3 downstream evaluation at 30% sparsity. $\pm$: 3-seed mean $\pm$ std ($s \in \{42, 123, 456\}$); TEAL is deterministic (single run). ARC-c exhibits high seed variance ($\sigma=6.4$ pp).

Method	PPL↓	ARC-c	WG	HS	PIQA	MMLU	GSM8K
Dense	5.52	60.2	78.7	83.2	81.9	62.4	36.5
KL 30%	6.93$\pm$0.19	44.9$\pm$6.4	70.2$\pm$1.2	69.7$\pm$0.8	74.8$\pm$0.4	43.3 $\pm$ 1.5	8.6 $\pm$ 2.2
TEAL 30%	8.38	41.1	63.5	60.1	72.1	43.5	15.9

Q SPON Orthogonality

SPON (Xu et al., 2025) bias correction is weakly correlated with the KL predictor (cosine similarity 0.196, $R^2=0.038$) and yields only 2.4% PPL improvement (12.08 $\rightarrow$ 11.79); the predictor already captures most recoverable signal. The decomposition confirms that SPON and the KL predictor operate on largely orthogonal error sources.

R Per-Layer KL Downstream Decomposition

Table 18 reports raw downstream numbers for the signal type vs. coupling scope decomposition at 30% sparsity (§4.5). BoolQ’s $\approx 100\%$ coupling is an upper bound confounded by high cross-seed variance (± 11.5 pp for per-layer KL); the $K=4$ pilot (Table 19) suggests much of this gap is train–inference mismatch.

Table 18: 2D decomposition at 30% sparsity (LLaMA-3.1-8B, uniform allocation). All rows are 3-seed mean $\pm$ std. Signal type = per-layer KL – MSE; coupling = E2E KL – per-layer KL. Percentages are upper bounds (dimensions not fully orthogonal; see §4.5).

	PPL↓	ARC-c	WG	MMLU	GSM8K	BoolQ
E2E KL	8.81 $\pm$ 0.02	43.7	68.9	40.7	7.5	73.7
Per-layer KL	9.05 $\pm$ 0.02	39.9	65.9	38.6	5.8	64.5
MSE	18.52 $\pm$ 1.68	35.8 $\pm$ 4.1	59.8 $\pm$ 2.6	30.7 $\pm$ 2.6	2.3 $\pm$ 0.7	64.5 $\pm$ 5.3
Total gap	9.71	7.9	9.1	10.0	5.2	9.2
Signal type (%)	97.5	51.9	67.0	79.0	67.3	≈ 0
Coupling (%)	2.5	48.1	33.0	21.0	32.7	≈ 100

S $K=4$ Mismatch Control

Table 19 reports 3-seed $K=4$ per-layer KL results at 30% sparsity. Training with $K=4$ random layers per step reduces the train–inference mismatch (4/32 vs. 1/32 layers masked during training, while inference applies all 32). The gap closure column reports the fraction of the $K=1 \rightarrow$ E2E KL gap closed by $K=4$.

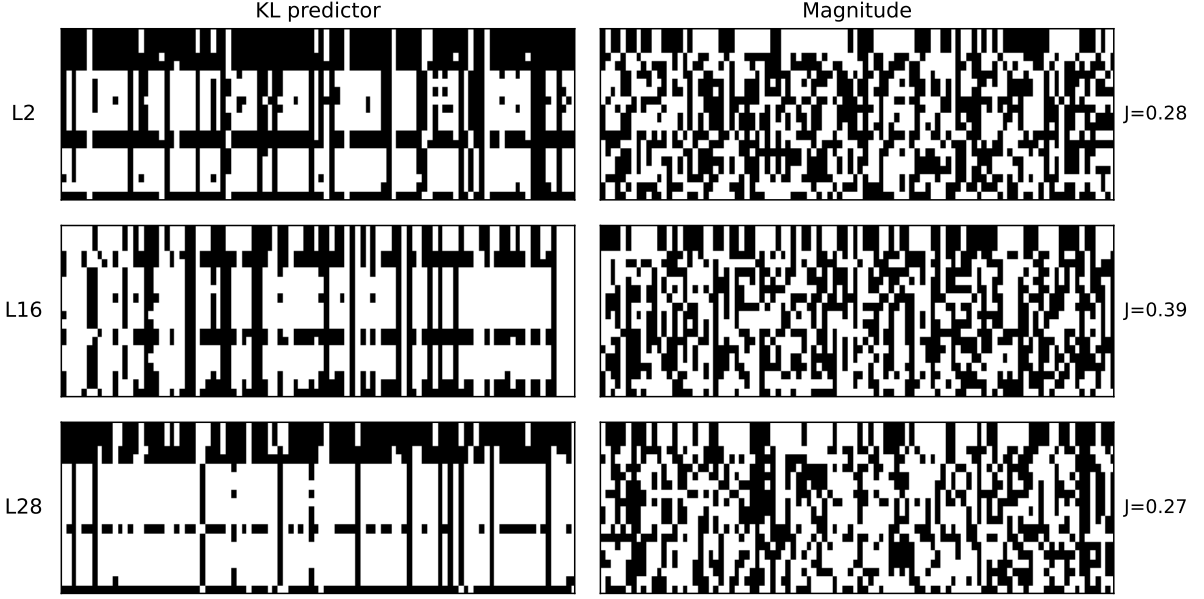


Figure 9: Binary mask comparison between KL-derived and magnitude-based masks at selected layers (L2, L16, L28) for 30% and 50% sparsity (64 samples). Black = selected by both, light regions = selected by only one method. Deep layers show the largest divergence (L28 Jaccard distance: 0.60 at 30%, 0.74 at 50%).

Table 19: 3-seed $K=4$ mismatch control at 30% sparsity (LLaMA-3.1-8B, uniform allocation, $s \in \{42, 123, 456\}$). Actual sparsity 29.6%; per-layer sparsity std=0.003 (near-uniform). Gap closure = $(K=4 - K=1)/(E2E - K=1) \times 100$.

	$K=4$ (3-seed)	$K=1$ (3-seed)	E2E KL (3-seed)	Gap closure (%)
PPL↓	8.89±0.01	9.05±0.02	8.81±0.02	66.7
ARC-c	40.0±1.3	39.9	43.7±2.5	2.6
WG	66.9±0.2	65.9	68.9±0.4	33.3
MMLU*	42.7±1.4	38.6	40.7±1.3	>100
GSM8K	7.0±1.6	5.8	7.5±0.1	70.6
BoolQ†	67.7±7.5	64.5±11.5	73.7	34.8

*MMLU $K=4$ systematically exceeds E2E across all 3 seeds;

attributable to $K=4$ ’s near-uniform per-layer sparsity (std=0.003) matching uniform eval allocation. Independently confirmed: removing the predictor’s middle-layer valley yields MMLU=44.0% (Appendix T). †BoolQ exhibits high inter-seed variance for both $K=4$ (std=7.5) and $K=1$ (std=11.5); gap closure unreliable.

T Per-Layer Sparsity Constraint Analysis

To test the causal role of the KL predictor’s learned per-layer allocation, we evaluate two inference-time interventions at 30% overall sparsity (seed 42, uniform allocation baseline):

Protecting late layers (L22–L28) does not improve knowledge-intensive metrics despite reducing their sparsity from 48–57% to $\leq 30\%$, while worsening PPL by 5.2%. Conversely, removing the protection valley (forcing L14–L18 to 30% instead of 14–18%) improves MMLU by +3.4pp and

Table 20: Per-layer sparsity constraint analysis. *Protect late*: cap L22–L28 sparsity $\leq 30\%$; *Remove valley*: override L14–L18 sparsity to 30%. Budget redistributed to maintain 30% overall.

Condition	PPL	ARC-c	MMLU	GSM8K
KL baseline (s42)	8.82	40.8	40.6	7.4
Protect late layers	9.26	42.4	40.7	6.5
Remove valley	8.76	39.3	44.0	9.0

GSM8K by +1.6pp at the cost of ARC-c -1.5 pp. The KL predictor’s learned allocation thus represents a task-dependent trade-off, prioritizing commonsense reasoning via middle-layer protection, rather than a uniformly optimal pattern. This allocation trade-off also explains the $K=4$ MMLU anomaly (§S): $K=4$ ’s near-uniform learned sparsity (std=0.003) inherently removes the valley, producing MMLU=43.5 comparable to the remove-valley condition (44.0).

U Per-Layer Sparsity Distribution

The KL predictor at 30% target sparsity learns a non-uniform per-layer allocation that differs markedly from magnitude-based selection. Aggregate sparsity is similar across broad regions (early L0–L7: KL 28.1% vs. magnitude 27.5%; late L24–L31: 44.9% vs. 47.2%), but middle layers (L8–L23) exhibit high internal variance despite similar averages (28.2% vs. 27.3%). Table 21 highlights

three sub-ranges.

Table 21: Per-layer sparsity sub-ranges: KL predictor vs. magnitude (LLaMA-3.1-8B, 30%, uniform allocation). Ranges denote min–max across layers in each group.

Layer Range	KL (%)	Mag (%)	Δ (pp)
L8–L12 (high pruning)	32–36	25–26	+7 to +10
L14–L18 (protection valley)	14–18	28–30	–8 to –16
L22–L28 (aggressive pruning)	48–57	—	—

V C4 Training Data Control

To test whether WikiText-103’s narrow domain drives the knowledge-task regression (§4), we train the KL predictor on C4 (Raffel et al., 2020) (diverse web text) under identical hyperparameters (1000 steps, 30% uniform, s42).

Table 22: C4-trained vs. WikiText-trained KL predictor (LLaMA-3.1-8B, 30% uniform, s42).

Training Data	PPL↓	ARC-c	WG	MMLU	GSM8K	HS / PIQA
WikiText-103	8.82	40.78	68.59	40.6	7.35	70.85 / 74.81
C4	11.55	43.09	68.67	39.95	4.09	69.24 / 75.03

The C4-trained predictor achieves comparable MMLU (40.0% vs. 40.6%) but worse GSM8K (4.1% vs. 7.4%) and higher PPL (11.55 vs. 8.82), despite training on more diverse text. This rules out training data composition as the primary cause of knowledge-task regression and supports the hypothesis that forward KL’s mode-covering property structurally under-protects rare reasoning-critical activations.

W Per-Layer Coupling Breadth Analysis

We vary the number of layers coupled per training step (K) from 1 (pure per-layer) to 32 (end-to-end), evaluating all predictors under TEAL uniform allocation at 30% sparsity on LLaMA-3.1-8B.

Table 23: K-curve: coupling breadth vs. predictor quality (30%, TEAL uniform, LLaMA-3.1-8B). All values are 3-seed mean $\pm$ std (s42/s123/s456) except $K=1$ downstream (single seed). “—” = not evaluated.

K	PPL↓	ARC-c	WG	MMLU	GSM8K	HS	PIQA
1	9.05 $\pm$ 0.02	39.9	65.9	38.6	5.8	—	—
2	10.75 $\pm$ 0.15	39.7 $\pm$ 1.2	63.2 $\pm$ 1.7	38.3 $\pm$ 3.2	2.8 $\pm$ 0.6	66.5 $\pm$ 1.0	73.5 $\pm$ 0.7
4	8.89 $\pm$ 0.01	40.0 $\pm$ 1.2	66.9 $\pm$ 0.2	42.7 $\pm$ 1.4	7.0 $\pm$ 1.6	67.8 $\pm$ 1.0	74.5 $\pm$ 0.5
8	9.00 $\pm$ 0.06	40.7 $\pm$ 1.0	68.7 $\pm$ 1.3	40.4 $\pm$ 0.6	6.8 $\pm$ 1.3	69.0 $\pm$ 1.1	75.1 $\pm$ 1.1
16	8.67 $\pm$ 0.02	41.9 $\pm$ 1.1	68.5 $\pm$ 0.9	40.9 $\pm$ 2.9	8.5 $\pm$ 1.2	70.1 $\pm$ 0.4	75.0 $\pm$ 0.8
32 (E2E)	8.81 $\pm$ 0.02	43.7 $\pm$ 2.5	68.9 $\pm$ 0.4	40.7 $\pm$ 1.3	7.5 $\pm$ 0.1	—	—

PPL exhibits a non-monotonic U-shape (Figure 3): $K=2$ is worst (10.75 $\pm$ 0.15), likely

due to insufficient per-layer updates (~ 62 updates/layer vs. ~ 250 at $K=8$). $K=16$ achieves the lowest PPL (8.67 $\pm$ 0.02), marginally better than E2E (8.81 $\pm$ 0.02). $K=16$ MMLU exhibits high seed variance (std=2.9; s42=44.0, s123=40.0, s456=38.7), with the 3-seed mean (40.9) indistinguishable from E2E (40.7 $\pm$ 1.3). The signal type gap ($K=1$ KL 9.05 vs. per-layer MSE 18.52 $\pm$ 1.68) of 9.47 PPL exceeds the coupling breadth range within KL (best $K=16$ 8.67 vs. worst $K=2$ 10.75 = 2.08 PPL), reinforcing that output-distribution alignment, not coupling scope, is the primary quality lever for PPL at this sparsity level.

X Licenses

LLaMA-3.1-8B is released under the Llama 3.1 Community License Agreement. Mistral-7B-v0.3 is released under the Apache 2.0 License. Qwen-2.5-14B is released under the Apache 2.0 License. WikiText-103 and WikiText-2 (Merity et al., 2017) are released under the Creative Commons Attribution-ShareAlike License (CC BY-SA 3.0). All evaluation benchmarks (ARC, WinoGrande, MMLU, GSM8K, HellaSwag, PIQA) are publicly available under their respective licenses. Our code will be released under the MIT License upon acceptance.

Y Use of AI Assistants

We used Claude (Anthropic) as an AI coding and writing assistant throughout this work. Specifically:

- **Code:** drafting experiment scripts, data-processing pipelines, and analysis utilities; debugging and refactoring.
- **Writing:** drafting and revising paper sections, polishing prose, and reformatting LaTeX.
- **Literature:** summarizing related papers to inform related-work section.